

| GUÍA 10 :Navegación y estructura visual en aplicaciones

Sigla	Asignatura	Experiencia de Aprendizaje
DSY1105	Desarrollo de Aplicaciones Móviles	EA2: Desarrollando componentes visuales y funcionalidades nativas para aplicaciones móviles
Tiempo	Modalidad de Trabajo	Indicadores de logro
1 hora	Parejas	IL2.1, IL2.2 y IL2.3

Antecedentes generales

Esta actividad busca que implementes un flujo de navegación funcional entre pantallas utilizando Jetpack Compose y una arquitectura MVVM actualizada.

Su propósito es reforzar tu comprensión de estructuras declarativas mediante el uso de componentes como TopAppBar, NavigationDrawer, NavigationBar, NavHost y NavController, promoviendo una jerarquía visual clara, una navegación fluida y una correcta adaptación a distintos tamaños de pantalla. Además, deberás aplicar la estructura de carpetas según el estándar 2025, incorporando lógica en el ViewModel, rutas reutilizables y principios de composición modular.

Requerimientos para esta actividad

Para el desarrollo de esta actividad deberás disponer de:

- Proyecto Android Studio nuevo.
- Conexión a internet para consultar documentación oficial.
- Carpetas organizadas con el patrón de arquitectura MVVM.
- Repositorio GitHub activo.

Actividad

En esta actividad práctica, debes desarrollar una navegación funcional entre pantallas reutilizando componentes visuales de Jetpack Compose como TopAppBar, NavigationDrawer, NavigationBar y organizando lógicamente el código usando el patrón MVVM. La tarea busca que explores de manera autónoma la construcción de un flujo visual coherente, aplicando principios de jerarquía visual, separación de responsabilidades y



adaptación a distintos tamaños de pantalla. Además, se espera el uso del Modifier para el estilo visual y la correcta definición de rutas.

El trabajo se realizará en parejas, registrando el avance en GitHub y Trello como evidencia de colaboración efectiva.

◇ Parte 1 – Creación de rutas

1. Crea tu proyecto nuevo para implementar las rutas y navegación. Esta vez anexa un nuevo paquete llamado navigation donde implementarás el archivo de rutas de tu app.
2. Agrega las dependencias al archivo build.gradle.kts (app) y elimina otras

```
dependencies {  
  
    implementation("androidx.navigation:navigation-compose:2.7.7")  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0-rc02")  
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.6.2")  
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")  
    implementation(libs.androidx.core.ktx)  
    implementation(libs.androidx.lifecycle.runtime.ktx)  
    implementation(libs.androidx.activity.compose)  
    implementation(platform(dependencyProvider = libs.androidx.compose.bom))  
    implementation(libs.androidx.ui)  
    implementation(libs.androidx.ui.graphics)  
    implementation(libs.androidx.ui.tooling.preview)  
    implementation(libs.androidx.material3)  
    //implementation(libs.androidx.navigation.runtime.android)  
    //implementation(libs.androidx.navigation.compose.jvmstubs)  
    implementation(libs.androidx.material3.window.size.class1.android)  
    testImplementation(libs.junit)  
    androidTestImplementation(libs.androidx.junit)  
    androidTestImplementation(libs.androidx.espresso.core)  
    androidTestImplementation(platform(dependencyProvider = libs.androidx.compose.bom))  
    androidTestImplementation(libs.androidx.ui.test.junit4)  
    debugImplementation(libs.androidx.ui.tooling)  
    debugImplementation(libs.androidx.ui.test.manifest)  
}
```

3. Implementar el código de creación de urls, se recomienda usar el sealed class para mayor seguridad. Recuerda crearlo en el package de navigation



```
package com.example.ejemplo2.navigation
// Sealed class para definir rutas tipo-safe en la navegación
28 Usages 4 Inheritors
sealed class Screen(val route: String) {
    // -----
    // Rutas simples (sin argumentos): Usamos 'data object'
    // 'data object' es un singleton seguro de tipos, introducido en Kotlin 1.9+.
    // Si usas una versión anterior de Kotlin, usa 'object' en su lugar.
    // -----
    // Representa la ruta a la pantalla de Inicio.
    6 Usages
    data object Home : Screen(route = "home_page")
    4 Usages
    data object Profile : Screen(route = "profile_page")
    // Representa la ruta a la pantalla de Configuración.
    5 Usages
    data object Settings : Screen(route = "settings_page")
    // -----
    // Ejemplos de Rutas con Argumentos (No usados en este ejercicio, pero importante)
    // Usamos 'data class' para encapsular los argumentos.
    // Esto asegura que, al navegar, los argumentos correctos sean pasados y tipados.
    // -----

    /**
     * Ejemplo de una ruta a una pantalla de detalles que requiere un 'itemId'.
     *
     * @param itemId El ID del elemento a mostrar en la pantalla de detalles.
     */
    data class Detail(val itemId: String) : Screen(route = "detail_page/{itemId}") {
        // Función para construir la ruta final con el argumento.
        // Esto evita errores al crear la ruta string.
        fun buildRoute(): String {
            // Reemplaza el placeholder "{itemId}" en la ruta base con el valor real.
            return route.replace(oldValue = "{itemId}", newValue = itemId)
        }
    }
}

// Si tuvieras más argumentos, se agregarían a la data class y a la cadena de ruta.
// data class UserProfile(val userId: String, val userName: String?) :
//     AppDestinations("profile/{userId}?name={userName}") { ... }
```

4. Crea el archivo para encapsular los posibles eventos de navegación.

```
package com.example.ejemplo2.navigation

// Representa los distintos tipos de eventos de navegación
17 Usages 3 Inheritors
sealed class NavigationEvent {

    /**
     * Evento para navegar a un destino específico.
     * @param route La [AppDestinations] (objeto de ruta tipado) a la que navegar.
     * @param popUpToRoute La ruta de destino en la pila de navegación hasta la cual se debe hacer 'pop'.
     * Si es null, no se hace 'pop' a un destino específico.
     * @param inclusive Si 'true', la ruta especificada en [popUpToRoute] también se elimina de la pila.
     * @param singleTop Si 'true', evita múltiples copias del mismo destino en la parte superior
     * de la pila si ya está presente (útil para navegación de barra inferior/lateral).
     */
    4 Usages
    data class NavigateTo(
        val route: Screen, // Ahora recibe un objeto AppDestinations (más seguro).
        val popUpToRoute: Screen? = null, // También acepta un objeto AppDestinations.
        val inclusive: Boolean = false,
        val singleTop: Boolean = false
    ) : NavigationEvent()

    /**
     * Evento para volver a la pantalla anterior en la pila de navegación.
     */
    3 Usages
    object PopBackStack : NavigationEvent()

    /**
     * Evento para navegar "hacia arriba" en la jerarquía de la aplicación.
     * Generalmente es equivalente a [PopBackStack] a menos que se use un grafo de navegación
     * con una jerarquía padre-hijo definida.
     */
    2 Usages
    object NavigateUp : NavigationEvent()
}
```

- Ahora crea el archivo de manipulación de estos eventos con las vistas, este debe crearse en el paquete de viewmodels de tu proyecto.



```
package com.example.ejemplo2.viewmodels

import androidx.lifecycle.ViewModel
import com.example.ejemplo2.navigation.NavigationEvent
import com.example.ejemplo2.navigation.Screen
import kotlinx.coroutines.flow.MutableSharedFlow
import kotlinx.coroutines.flow.SharedFlow
import kotlinx.coroutines.flow.asSharedFlow
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch

8 Usages
class MainViewModel : ViewModel() {

    4 Usages
    private val _navigationEvents = MutableSharedFlow<NavigationEvent>()
    // 'receiveAsFlow()': Expone el Channel como un Flow de solo lectura para que la UI pueda observarlo.
    // Esto previene que la UI pueda enviar eventos al Channel directamente.
    1 Usage
    ⚠ val navigationEvents: SharedFlow<NavigationEvent> = _navigationEvents.asSharedFlow()
    // Función que emite el evento de navegación hacia la ruta deseada.
    5 Usages
    fun navigateTo(screen: Screen) {
        CoroutineScope(context = Dispatchers.Main).launch {
            _navigationEvents.emit(value = NavigationEvent.NavigateTo(route = screen))
        }
    }
    // - Función para volver atrás
    fun navigateBack() {
        CoroutineScope(context = Dispatchers.Main).launch {
            _navigationEvents.emit(value = NavigationEvent.PopBackStack)
        }
    }
    // † Función para navegar hacia arriba (padre)
    fun navigateUp() {
        CoroutineScope(context = Dispatchers.Main).launch {
            _navigationEvents.emit(value = NavigationEvent.NavigateUp)
        }
    }
}
```

◊ Parte 2 – Personalización visual de las pantallas e integración de menú

6. Crea tres pantallas en el paquete correspondiente con los componentes de compose que quieras. Puedes usar textos, botones, imágenes. Y no olvides incluir un TopBar en una de ellas y un BottomBar en la otra. Guíate de la documentación oficial para generarlos.

```

HomeScreen.kt x
3  import androidx.compose.foundation.layout.*
4  import androidx.compose.material.icons.Icons
5  import androidx.compose.material.icons.filled.Menu
6  import androidx.compose.material3.*
7  import androidx.compose.material3.Text
8  import androidx.compose.material3.Icon
9  import androidx.compose.material3.Button
10 import androidx.compose.material3.TopAppBar
11 import androidx.compose.material3.Scaffold
12 import androidx.compose.material3.ModalNavigationDrawer
13 import androidx.compose.material3.DrawerValue
14 import androidx.compose.material3.ModalDrawerSheet
15 import androidx.compose.material3.NavigationDrawerItem
16 import androidx.compose.runtime.*
17 import androidx.compose.ui.Alignment
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.unit.dp
20 import androidx.lifecycle.viewmodel.compose.viewModel
21 import androidx.navigation.NavController
22 import com.example.ejemplo2.navigation.Screen
23 import com.example.ejemplo2.viewmodels.MainViewModel
24 import kotlinx.coroutines.launch
25
26 3 Usages
27 @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
28 @Composable
29 fun HomeScreen(
30     navController: NavController,
31     viewModel: MainViewModel = viewModel()
32 ) {
33     val drawerState = rememberDrawerState(initialValue = DrawerValue.Closed)
34     val scope = rememberCoroutineScope()
35
36     ModalNavigationDrawer(
37         drawerState = drawerState,
38         drawerContent = {
39             ModalDrawerSheet {
40                 Text(text = "Menú", modifier = Modifier.padding(all = 16.dp))
41                 NavigationDrawerItem(
42                     label = { Text(text = "Ir a Perfil") },
43                     selected = false,
44                     onClick = {

```



```
44 ->         scope.launch { drawerState.close() }
45         viewModel.navigateTo(Screen.Profile)
46     }
47     )
48 }
49 }
50 ) {
51     Scaffold(
52         topBar = {
53             TopAppBar(
54                 title = { Text( text = "Pantalla Home" ) },
55                 navigationIcon = {
56                     IconButton(onClick = {
57 ->                 scope.launch { drawerState.open() }
58             }) {
59                 Icon( imageVector = Icons.Default.Menu, contentDescription = "Menú")
60             }
61         }
62     )
63 }
64 ) { innerPadding ->
65     Column(
66         modifier = Modifier
67             .padding( paddingValues = innerPadding)
68             .fillMaxSize(),
69         horizontalAlignment = Alignment.CenterHorizontally,
70         verticalArrangement = Arrangement.Center
71     ) {
72         Text( text = ";Bienvenido a la Página de Inicio (MVVM)!" )
73         Spacer(modifier = Modifier.height( height = 16.dp))
74         Button(onClick = { viewModel.navigateTo(Screen.Settings) }) {
75             Text( text = "Ir a Configuración" )
76         }
77     }
78 }
79 }
80 }
```



```
ProfileScreen.kt x
16 @OptIn( ...markerClass = ExperimentalMaterial3Api::class)
17 @Composable
18 fun ProfileScreen(
19     navController: NavController,
20     viewModel: MainViewModel = viewModel()
21 ) {
22     val items = listOf(Screen.Home, Screen.Profile)
23     var selectedItem by remember { mutableStateOf( value = 1) }
24
25     Scaffold(
26         bottomBar = {
27             NavigationBar {
28                 items.forEachIndexed { index, screen ->
29                     NavigationBarItem(
30                         selected = selectedItem == index,
31                         onClick = {
32                             selectedItem = index
33                             viewModel.navigateTo(screen)
34                         },
35                         label = { Text( text = screen.route ) },
36                         icon = {
37                             Icon(
38                                 imageVector = if (screen == Screen.Home) Icons.Default.Home else Icons.Default.Person,
39                                 contentDescription = screen.route
40                             )
41                         }
42                     )
43                 }
44             }
45         }
46     ) { innerPadding ->
47         Column(
48             modifier = Modifier
49                 .padding( paddingValues = innerPadding)
50                 .fillMaxSize(),
51             horizontalAlignment = Alignment.CenterHorizontally,
52             verticalArrangement = Arrangement.Center
53         ) {
54             Text( text = "¡Bienvenido al Perfil!")
55         }
56     }
57 }
```

```
SettingsScreen.kt
12 import com.example.myapplication.viewmodel.MainViewModel
13
14 2 Usages
15 @Composable
16 fun SettingsScreen(
17     navController: NavController, // Controlador de navegación para moverse entre pantallas
18     viewModel: MainViewModel // ViewModel que centraliza la navegación (eventos)
19 ) {
20     // Estructura visual centralizada
21     Column(
22         modifier = Modifier
23             .fillMaxSize() // Ocupar todo el alto disponible
24             .padding(all = 16.dp), // Margen interno general
25         verticalArrangement = Arrangement.Center, // Centrar elementos verticalmente
26         horizontalAlignment = Alignment.CenterHorizontally // Centrar elementos horizontalmente
27     ) {
28         // Título o texto principal
29         Text(text = "Pantalla de Configuración (Settings)")
30
31         Spacer(modifier = Modifier.height(height = 24.dp)) // Espacio vertical
32
33         // Botón para volver al Home
34         Button(
35             onClick = {
36                 viewModel.navigateTo(Screen.Home) // Emitir evento de navegación al ViewModel
37             }
38         ) {
39             Text(text = "Volver al Inicio")
40         }
41
42         Spacer(modifier = Modifier.height(height = 16.dp)) // Más espacio
43
44         // Botón para ir al Perfil
45         Button(
46             onClick = {
47                 viewModel.navigateTo(Screen.Profile) // Emitir evento para ir a perfil
48             }
49         ) {
50             Text(text = "Ir a Perfil")
51         }
52     }
53 }
```

- Adapta el mainActivity para mostrar la página principal y puedas probar el funcionamiento.

```
MainActivity.kt x
3      import android.os.Bundle
4      import androidx.activity.ComponentActivity
5      import androidx.activity.compose.setContent
6      import androidx.compose.foundation.layout.fillMaxSize
7      import androidx.compose.foundation.layout.padding
8      import androidx.compose.material3.*
9      import androidx.compose.runtime.*
10     import androidx.lifecycle.viewmodel.compose.viewModel
11     import androidx.navigation.compose.NavHost
12     import androidx.navigation.compose.composable
13     import androidx.navigation.compose.rememberNavController
14     import com.example.ejemplo2.navigation.NavigationEvent
15     import com.example.ejemplo2.navigation.Screen
16     import com.example.ejemplo2.ui.theme.HomeScreen
17     import com.example.ejemplo2.ui.theme.ProfileScreen
18     import com.example.ejemplo2.ui.theme.SettingsScreen
19     import com.example.ejemplo2.ui.theme.Ejemplo2Theme
20     import com.example.ejemplo2.viewmodels.MainViewModel
21     import kotlinx.coroutines.flow.collectLatest
22     import androidx.compose.ui.Modifier
23     1 Usage
24     @OptIn(markerClass = ExperimentalMaterial3Api::class)
25     class MainActivity : ComponentActivity() {
26         override fun onCreate(savedInstanceState: Bundle?) {
27             super.onCreate(savedInstanceState)
28             setContent {
29                 Ejemplo2Theme {
30                     // ViewModel y NavController
31                     val viewModel: MainViewModel = viewModel()
32                     val navController = rememberNavController()
33
34                     // Escuchar eventos de navegación emitidos por el ViewModel
35                     LaunchedEffect(key1 = Unit) {
36                         viewModel.navigationEvents.collectLatest { event ->
37                             when (event) {
38                                 is NavigationEvent.NavigateTo -> {
39                                     navController.navigate(route = event.route.route) {
40                                         event.popUpToRoute?.let {
41                                             popUpTo(route = it.route) {
42                                                 inclusive = event.inclusive
43                                             }
44                                         }
45                                     }
46                                 }
47                             }
48                         }
49                     }
50                 }
51             }
52         }
53     }
```

```
44         }
45         launchSingleTop = event.singleTop
46         restoreState = true
47     }
48 }
49
50 is NavigationEvent.PopBackStack -> navController.popBackStack()
51 is NavigationEvent.NavigateUp -> navController.navigateUp()
52 }
53 }
54 }
55
56 // Layout base con NavHost
57 Scaffold(
58     modifier = Modifier.fillMaxSize()
59 ) { innerPadding ->
60
61     NavHost(
62         navController = navController,
63         startDestination = Screen.Home.route,
64         modifier = Modifier.padding( paddingValues = innerPadding)
65     ) {
66         composable(route = Screen.Home.route) {
67             HomeScreen(navController = navController, viewModel = viewModel)
68         }
69         composable(route = Screen.Profile.route) {
70             ProfileScreen(navController = navController, viewModel = viewModel)
71         }
72         composable(route = Screen.Settings.route) {
73             SettingsScreen(navController = navController, viewModel = viewModel)
74         }
75     }
76 }
77 }
78 }
79 }
80 }
```

◇ Parte 3 – Evidencia de trabajo colaborativo

- Realizar un commit con mensaje: "Navegación y menú generado".
- En Trello: actualizar la tarjeta de tarea correspondiente.
- Crear una rama si se está trabajando por feature (feature/home-screen).
- Adjuntar captura del resultado visual al tablero.